INGENIERÍA DEL SOFTWARE I · UCM · CURSO 2026/27

Better

Web de apuestas deportivas. Guía completa del proyecto: qué se ha hecho, cómo está organizado y por qué

Equipo: Carlos Martín-Salas · Jaime Martín · David Ortega · Jing Li · Carlos Jurado

Product Owner: Gonzalo Rubén Méndez Pozo · **Scrum Master:** Carlos Jurado

Repositorio: riverri/IS1 · **Web:** is1-1.onrender.com

Estado descrito: 19 de octubre de 2026

Índice

Introducción

0. Cómo leer esta guía

Parte I · El proyecto

1. De qué va el proyecto
2. El proyecto en cifras

Parte II · Conceptos básicos

3. Cómo funciona una aplicación web
4. Las tecnologías, una a una
5. La arquitectura por capas

Parte III · El repositorio por dentro

6. Las carpetas del repositorio
7. Los paquetes del código, clase a clase
8. El modelo de datos
9. Las páginas y sus direcciones
10. La configuración

Parte IV · Cómo funciona cada cosa

11. Usuarios, sesión y moneditas
12. El algoritmo de cuotas
13. Hacer una apuesta simple, paso a paso
14. El boleto y las combinadas
15. Resolver las apuestas
16. Otros mercados de fútbol
17. Apuestas a largo plazo
18. El panel del creador
19. Estadísticas de equipos
20. Ranking, perfiles, ligas y avisos
21. Juego responsable y baja de la cuenta
22. Datos reales con la API
23. Seguridad

Parte V · Base de datos

24. Flyway y las migraciones V1–V10

Parte VI · Calidad

25. Las pruebas automáticas
26. La matriz de trazabilidad
27. La integración continua

Parte VII · Despliegue

28. Ejecutarlo en tu ordenador
29. Compartirlo con un túnel
30. El servidor permanente

Parte VIII · El proceso Scrum

31. Roles, eventos y artefactos
32. Product Backlog e historias de usuario
33. Los doce sprints
34. Métricas

Parte IX · Historia y revisión

35. Cronología del desarrollo
36. IS1 e IS1-Copy
37. La revisión: 34 fallos

Parte X · Para el día a día

38. Qué queda por hacer
39. Cómo hacer un cambio
40. Preguntas que te pueden hacer
41. Glosario

0 Cómo leer esta guía

Esta guía explica el proyecto desde cero. No hace falta saber desarrollo web para seguirla: las palabras técnicas se explican la primera vez que aparecen y al final hay un glosario.

Está ordenada de lo general a lo concreto:

- **Partes I y II:** qué es la aplicación y las ideas básicas para entender cómo está hecha. Si solo tienes una hora, lee estas dos.
- **Partes III y IV:** dónde está cada cosa en el código y cómo funciona por dentro. Son las que hay que tener a mano al tocar código.
- **Partes V a VII:** base de datos, pruebas y despliegue.
- **Partes VIII y IX:** cómo se ha organizado el trabajo con Scrum, la historia del proyecto y la revisión de fallos.
- **Parte X:** lo que queda pendiente, cómo hacer un cambio y preguntas típicas de una defensa.

Convenciones

- Los nombres de clases, archivos y direcciones van en `letra de código`.
- Las rutas de código se abrevian: `apuesta/ApuestaService.java` es `src/main/java/es/ucm/fdi/is1/apuestas/apuesta/ApuestaService.java`.
- **HU-23** es la historia de usuario 23 (ver la Parte VIII); **#27** es el issue 27 de GitHub; **F-05** y **N-12** son fallos de la revisión (sección 37).

El proyecto

1. De qué va el proyecto

Es el proyecto de la asignatura **Ingeniería del Software I**. El objetivo no es solo programar una aplicación, sino hacerlo como se hace en un equipo profesional: con requisitos escritos como historias de usuario, un proceso ágil (**Scrum**), control de versiones con Git y GitHub, pruebas automáticas y documentación.

La aplicación es una **web de apuestas deportivas con moneditas virtuales**. No se usa dinero real en ningún momento: cada jugador recibe moneditas al registrarse y recargas gratuitas cada semana.

Quién usa la aplicación

Actor	Qué puede hacer
Visitante (sin cuenta)	Ver el catálogo de partidos con sus cuotas, las fichas de los equipos, los mercados a largo plazo, el ranking y los perfiles públicos. Para apostar tiene que registrarse.
Jugador (rol `USUARIO`)	Apostar (simples, combinadas y a largo plazo), seguir sus apuestas, cancelarlas o cambiar el importe antes de que empiece el partido, ver su historial y estadísticas, recibir avisos, competir en el ranking y en ligas privadas, ponerse límites y eliminar su cuenta.
Creador de apuestas (rol `CREADOR`)	Es el administrador. Da de alta competiciones, equipos, jugadores y partidos; introduce resultados (que resuelven y pagan las apuestas); suspende, anula o corrige eventos; gestiona los mercados a largo plazo; fija límites de apuesta y ve las cuentas de la casa. **No puede apostar.**

Qué tiene la aplicación

- **Cuotas propias:** no se copian de ninguna casa de apuestas; las calcula un algoritmo a partir de la calidad de los equipos, su forma reciente y el dinero apostado (sección 12).
- **Tres tipos de apuesta:** simple (un pronóstico), combinada (varios pronósticos de partidos distintos; se multiplican las cuotas) y a largo plazo (Balón de Oro, campeón de liga…).
- **Más mercados en fútbol:** además de 1X2, doble oportunidad, más/menos de 2,5 goles y ambos marcan.
- **Resolución automática:** al introducir el resultado, todas las apuestas se resuelven y se pagan solas.
- **Datos reales:** partidos, resultados y plantillas de LaLiga y la Champions desde la API de football-data.org.
- **Banco de estadísticas:** ficha de cada equipo con resultados, racha, gráfico de evolución, clasificación, plantilla con notas y alineación probable, y el cara a cara de los rivales.
- **Parte social:** ranking general, perfiles públicos y ligas privadas con código de invitación.
- **Juego responsable:** límites diarios y semanales, pausas y baja de la cuenta.

- **Deportes:** fútbol, baloncesto, tenis, Fórmula 1 y MotoGP (los cuatro últimos con eventos de ejemplo).

2. El proyecto en cifras

12 sprints	**214** puntos de historia completados	**45** historias de usuario terminadas (de 53 escritas)
276 pruebas automáticas en 45 clases	**144** clases Java (≈ 9.300 líneas)	**10** migraciones de la base de datos
32 plantillas HTML	**94** criterios de aceptación en las historias hechas, 91 con prueba automática	**34** fallos encontrados en la revisión, todos corregidos

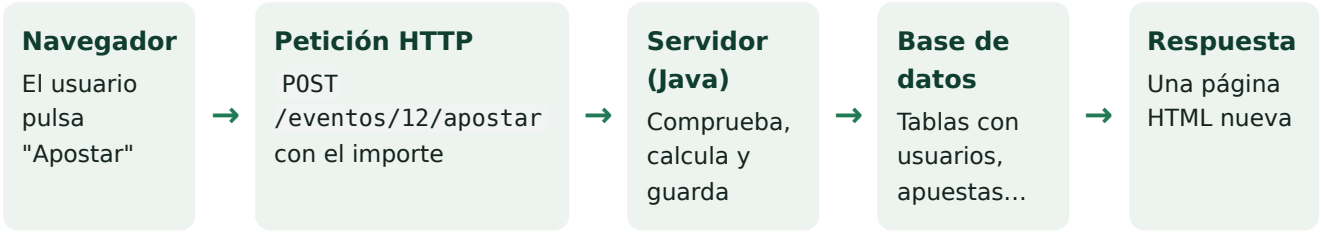
II Conceptos básicos

Antes de mirar el código conviene entender cuatro ideas: qué es un cliente y un servidor, qué es una petición HTTP, qué hace cada tecnología y cómo se reparte el código en capas.

3. Cómo funciona una aplicación web

Cliente y servidor

Una aplicación web tiene dos partes. El **cliente** es el navegador (Chrome, Firefox, el del móvil): enseña páginas y envía lo que el usuario hace. El **servidor** es un programa que está siempre encendido esperando peticiones: en nuestro caso, la aplicación Java. El servidor guarda los datos en una **base de datos**.



HTTP, GET y POST

El navegador y el servidor hablan con el protocolo **HTTP**. Cada petición tiene un **método** y una **dirección** (URL):

- **GET** pide ver algo y no debe cambiar nada (por ejemplo, `GET /ranking`). Es lo que pasa al escribir una dirección o pulsar un enlace.
- **POST** envía datos que cambian algo (apostar, registrarse, introducir un resultado). Es lo que hace un formulario con `method="post"`.

Que GET no cambie nada es importante: el navegador puede repetir o precargar peticiones GET por su cuenta. Uno de los fallos de la revisión (F-11) era justo un enlace GET que añadía cosas al boleto.

HTML, CSS y JavaScript

HTML describe el contenido de la página (títulos, tablas, formularios); **CSS**, su aspecto (colores, tamaños, cómo se adapta al móvil); **JavaScript**, el comportamiento en el navegador. En este proyecto casi todo el trabajo lo hace el servidor: genera el HTML ya completo y el JavaScript solo se usa para detalles, como calcular la ganancia potencial mientras se escribe el importe o pedir confirmación antes de borrar.

Sesión y cookies

HTTP no recuerda nada entre peticiones. Para saber quién es el usuario, al iniciar sesión el servidor crea una **sesión** (un espacio de memoria para ese usuario) y el navegador guarda un identificador en una **cookie**, que envía en cada petición. Así el servidor sabe que la petición viene de un usuario concreto. En la sesión también se guarda el **boleto** que el jugador está montando.

4. Las tecnologías, una a una

La decisión de qué tecnologías usar está escrita en `docs/decisiones/0001-stack-tecnologico.md`. Se eligió **Java** porque es el lenguaje que conoce todo el equipo, y el resto se eligió por ser lo más estándar alrededor de Java.

Tecnología	Para qué sirve en el proyecto
Java 21	El lenguaje de toda la parte del servidor.
Spring Boot 4	Un *framework*: un esqueleto que ya resuelve lo común de una aplicación web (recibir peticiones, conectar con la base de datos, seguridad…). Nosotros solo escribimos lo propio de las apuestas. Arranca un servidor web dentro de la aplicación, así que basta con ejecutar un programa Java.
Thymeleaf	Plantillas HTML con huecos que se rellenan con datos de Java. Por ejemplo, `<span th:text="${usuario.saldo}">` se convierte en el saldo real. Están en `src/main/resources/templates`.
Spring Data JPA / Hibernate	Traduce entre objetos Java y tablas de la base de datos (es un **ORM**). Una clase con `@Entity`, como `Apuesta`, corresponde a una tabla, y sus atributos a columnas. Un **repositorio** (`ApuestaRepository`) da métodos para buscar y guardar sin escribir SQL.
H2	Base de datos que se ejecuta dentro de la propia aplicación y guarda los datos en un archivo (`datos/apuestas.mv.db`). Es la que se usa en cada ordenador.
PostgreSQL	Base de datos "de verdad", la que se usa en el servidor de Internet (en Neon).
Flyway	Crea y actualiza las tablas con scripts SQL numerados (`V1`, `V2`…), aplicándolos en orden una sola vez. Así una base de datos antigua se actualiza sola sin perder datos (Parte V).
Spring Security	Inicio y cierre de sesión, contraseñas cifradas (BCrypt), qué páginas puede ver cada rol y protección CSRF.
Maven	Descarga las librerías (dependencias, en `pom.xml`), compila, ejecuta las pruebas y empaqueta. Se usa con `./mvnw` (Windows: `mvnw.cmd`), que funciona sin instalar Maven.
JUnit 5 y MockMvc	Pruebas automáticas. MockMvc simula peticiones HTTP sin abrir un navegador.
Git y GitHub	Control de versiones: cada cambio queda guardado (commit) con su autor y su descripción. En GitHub están el repositorio, los **issues** (tareas), los **pull requests** (propuestas de cambio que se revisan antes de juntarlas con `main`) y el tablero.
GitHub Actions	Integración continua (CI): en cada pull request compila y ejecuta todas las pruebas, con H2 y con PostgreSQL.
Docker	Empaqueta la aplicación con todo lo que necesita en una "imagen" que se ejecuta igual en cualquier servidor. La usa Render (`Dockerfile`).

5. La arquitectura por capas

El código de cada funcionalidad se reparte en **capas**, y cada capa solo habla con la de abajo. Es el patrón **MVC** (Modelo-Vista-Controlador) con una capa de servicios en medio:

Vista · plantillas Thymeleaf (`templates/*.html`). Solo muestran datos; no deciden nada.

↓ ↑

Controlador · clases `...Controller` . Reciben la petición HTTP, leen el formulario, llaman al servicio y eligen qué plantilla devolver. Ejemplo: `ApuestaController` .

↓ ↑

Servicio · clases `...Service`. Aquí están las **reglas del negocio**: comprobar el saldo y los límites, calcular la cuota, resolver una apuesta. Abren las **transacciones** (`@Transactional`): o se guarda todo o nada.

↓ ↑

Modelo · entidades (`@Entity`) y repositorios. Las entidades también tienen lógica propia: por ejemplo, `Apuesta.reevaluar()` decide si está ganada o perdida.

↓ ↑

Base de datos · H2 o PostgreSQL, con las tablas creadas por Flyway.

Además, el código se agrupa **por funcionalidad** y no por tipo: todo lo de las apuestas está en el paquete `apuesta` , todo lo de los usuarios en `usuarios` , etc. Así, para tocar algo de las ligas, todo está en la carpeta `ligas` .

Inyección de dependencias

Ninguna clase hace `new ApuestaService(...)` . Spring crea un único objeto de cada clase marcada con `@Service` , `@Controller` , `@Component` … y se lo pasa por el constructor a quien lo necesite. Por eso los constructores reciben repositorios y servicios como parámetros. También permite cambiar piezas en las pruebas: por ejemplo, el reloj (`Clock`) se inyecta para poder fijar la hora en una prueba.

III El repositorio por dentro

6. Las carpetas del repositorio

```
IS1/
├── README.md                   Portada del proyecto: qué es, cómo ejecutarlo, funcionalidades
├── CONTRIBUTING.md             Cómo trabajar con ramas, commits y pull requests
├── pom.xml                     Configuración de Maven: versión de Java y librerías
├── mvnw, mvnw.cmd, .mvn/       Maven "incluido": ./mvnw funciona sin instalar nada
├── Dockerfile, .dockerignore   Cómo empaquetar la aplicación para el servidor (Render)
├── render.yaml                 Configuración del servicio en Render
├── compartir.bat / .sh         Lanzan la aplicación y un túnel para jugar desde otros equipos
├── .github/
│   ├── workflows/ci.yml        Integración continua: compila y pasa las pruebas en cada PR
│   ├── ISSUE_TEMPLATE/         Plantillas de issue: historia de usuario, tarea y fallo
│   └── pull_request_template.md
├── docs/
│   ├── requisitos/             vision.md · historias-de-usuario.md · trazabilidad.md
│   ├── scrum/                  proceso.md · product-backlog.md · metricas.md · sprints/ ·
graficas/
│   ├── diseno/uml.md           Diagramas: casos de uso, clases, estados, secuencia, datos
│   ├── decisiones/             0001 (tecnologías) y 0002 (saldo negativo)
│   └── recursos.md             Enlaces útiles para aprender
└── src/
    ├── main/
    │   ├── java/es/ucm/fdi/is1/apuestas/   El código, en 12 paquetes (sección 7)
    │   └── resources/
    │       ├── application*.properties     Configuración (sección 10)
    │       ├── db/migration/h2/            Scripts de la base de datos para H2
    │       ├── db/migration/postgresql/    Los mismos para PostgreSQL
    │       ├── templates/                  Las 30 plantillas HTML
    │       └── static/css/estilos.css      Todo el diseño visual
    └── test/java/...                       Las pruebas, con los mismos paquetes
```

Hay dos carpetas que se crean solas y **no se suben a GitHub** (están en `.gitignore`): `target/`, con el programa compilado, y `datos/`, con la base de datos H2 de tu ordenador.

7. Los paquetes del código, clase a clase

El punto de entrada es `ApuestasApplication.java`, con el método `main` que arranca Spring Boot. El resto está repartido en estos paquetes:

Paquete	Qué contiene
`usuarios`	Cuentas, roles, registro, inicio de sesión, saldo y recargas, límites personales, sesiones.
`equipos`	Competiciones, equipos, jugadores, plantillas y alineación probable.
`eventos`	Partidos (eventos), catálogo, búsqueda, ficha de equipo, cara a cara y gráfico de evolución.
`cuotas`	El algoritmo de cuotas y los tipos de resultado.

`apuesta`	Apuestas y selecciones, boleto, resolución, cancelación, límites de la casa, ranking, estadísticas, cuentas de la casa y baja de la cuenta. Es el paquete central.
`mercados`	Mercados a largo plazo (Balón de Oro, campeón…) y sus candidatos.
`ligas`	Ligas privadas con código de invitación.
`notificaciones`	Avisos dentro de la web cuando se resuelve una apuesta.
`gestion`	El panel del creador: formularios y controladores de alta y edición.
`api`	Conexión con football-data.org y sincronización de partidos y plantillas.
`config`	Configuración general: seguridad, reloj y datos iniciales.
`web`	Piezas comunes de la web: página de inicio, formato de fechas, colores de escudos, avisos de concurrencia.

Dependencias entre paquetes

`apuesta` usa `eventos`, `cuotas`, `mercados`, `usuarios` y `notificaciones`; `ligas` usa `apuesta` (para el ranking). Se evita que dos paquetes dependan el uno del otro (un **ciclo**): por eso, cuando un usuario se da de baja, `usuarios` no llama a `ligas`, sino que publica un evento (`CuentaEliminada`) que `ligas` escucha.

Tipos de clase que vas a encontrar

Sufijo o anotación	Qué es
`@Entity`	Una tabla de la base de datos (sección 8).
`...Repository`	Interfaz para leer y guardar una entidad. Spring escribe el código solo, a partir del nombre de los métodos (`findByEmail`) o de una consulta `@Query`.
`...Service`	Reglas del negocio y transacciones.
`...Controller`	Atiende las peticiones de unas direcciones concretas.
`...Form`	Los campos de un formulario, con sus validaciones (`@NotBlank`, `@Size`, `@DecimalMin`…).
`...Exception`	Un error del negocio con nombre propio (saldo insuficiente, evento no disponible…). Las de "no encontrado" devuelven una página 404.
`record`	Una clase solo de datos, sin lógica, para pasar información a las plantillas (`FichaEquipo`, `PuestoRanking`, `CuentasCasa`…).
`enum`	Una lista cerrada de valores: estados, deportes, resultados, roles.

Paquete `usuarios`

Clase	Qué hace

`Usuario` (entidad)	Email, nombre, contraseña cifrada, rol, saldo, última recarga, límites, pausa y si está eliminado. Métodos para cargar (apostar), abonar (cobrar) y ajustar el saldo, y para recargar.
`Rol`	`USUARIO` (jugador) o `CREADOR`.
`UsuarioService`	Registro, recarga gratuita al consultar, cambio de nombre y contraseña, límites, pausas y anonimizar al dar de baja.
`UsuariosController`	Páginas `/registro`, `/login` y `/cuenta`.
`UsuarioDetailsService`	Le dice a Spring Security cómo buscar un usuario por email al iniciar sesión.
`RecargaAlEntrarHandler`	Al iniciar sesión, aplica la recarga gratuita si toca.
`SaldoProperties`	Lee la bienvenida (1.000), la recarga (200) y el periodo (7 días) de la configuración.
`IntentosLogin`, `Sesiones`, `Contrasenas`, `CuentaEliminada`	Añadidas en la revisión: bloqueo tras 5 fallos, cierre de las demás sesiones, límite de 72 bytes de la contraseña y evento de baja.
Formularios	`RegistroForm`, `NombreForm`, `PasswordForm`, `LimitesPersonalesForm`.

Paquete `equipos`

Clase	Qué hace
`Competicion`, `Equipo`, `Jugador`	Entidades. Un equipo tiene deporte, **calidad** (0–10), **forma**, escudo, competiciones e identificador de la API. Un jugador tiene posición, dorsal, nacionalidad y **nota** (6,0 al empezar).
`Deporte`, `Forma`, `Posicion`	Enumeraciones. La forma va de "muy mala racha" (−2) a "muy buena racha" (+2).
`EquiposService`	Listado de equipos por deporte y competición, con buscador.
`PlantillaService`	Añadir, puntuar y quitar jugadores (HU-50).
`Alineacion`	Calcula la alineación probable en 4-3-3: por cada posición, los de mejor nota.
`Nombres`	Normaliza nombres para comparar ("Atlético" y "Atletico").

Paquete `eventos`

Clase	Qué hace
`Evento` (entidad)	Un partido o carrera: competición, local, visitante, fecha y hora, estado, resultado, goles y fase. Decide si admite apuestas (programado y sin empezar).
`EstadoEvento`	`PROGRAMADO`, `SUSPENDIDO`, `FINALIZADO`, `ANULADO`.

`CatalogoService`, `EventosController`	Catálogo por deporte, búsqueda, página del partido con sus cuotas y cara a cara.
`FichaEquipoService`	Construye la ficha del equipo: últimos 10 resultados, racha de 5, balance, clasificación y próximos 5 partidos.
`FichaEquipo`, `CaraACara`, `GraficoEvolucion`, `Balance`	Datos que se muestran en esas páginas. `GraficoEvolucion` dibuja el gráfico de puntos acumulados en SVG.

Paquete `cuotas`

Clase	Qué hace
`CalculadoraCuotas`	El algoritmo (sección 12): cuotas 1X2 y de los otros mercados de fútbol.
`Cuotas`	Las tres cuotas de un evento (el empate es `null` si el deporte no admite empate).
`Resultado`	`LOCAL`, `EMPATE`, `VISITANTE`.
`Especial`	Los otros mercados: 1X, X2, 12, +2,5, −2,5, ambos marcan sí/no.
`VolumenApostado`	Interfaz para saber cuánto dinero hay en cada resultado (la implementa `apuesta/VolumenApuestas`).

Paquete `apuesta`

Clase	Qué hace
`Apuesta` (entidad)	Usuario, importe, cuota total, estado, lo ya pagado y fecha, con sus selecciones. Sabe resolverse, cancelarse y cambiar de importe.
`Seleccion` (entidad)	Un pronóstico dentro de una apuesta: evento y resultado, o evento y mercado especial, o candidato de un mercado a largo plazo. Guarda la cuota del momento de apostar.
`EstadoApuesta`, `EstadoSeleccion`	Apuesta: `ACTIVA`, `GANADA`, `PERDIDA`, `ANULADA`, `CANCELADA`. Selección: `PENDIENTE`, `ACERTADA`, `FALLADA`, `ANULADA`.
`ApuestaService`	Apostar (simple, combinada, largo plazo), cancelar, cambiar el importe, historial, activas y estadísticas.
`ApuestaController`	Página del partido para apostar y `/apuestas` (mis apuestas).
`Boleto`, `BoletoService`, `BoletoController`, `BoletoVista`	El boleto que el jugador va montando (vive en su sesión), con su cuota total y la confirmación.
`ResolucionService`	Introducir resultado o marcador, suspender, reactivar y anular eventos, y el dinero apostado a cada resultado.
`ResolucionMercadosService`	Lo mismo para los mercados a largo plazo.

`Limites` , `LimitesService`	Límites de la casa: importe mínimo (1), máximo (500) y selecciones por combinada (10), y el interruptor del juego responsable.
`VolumenApuestas`	Suma el dinero de las apuestas activas por resultado, para el ajuste de cuotas.
`RankingService` , `CriterioRanking` , `PuestoRanking`	Ranking por saldo, ganancias o % de aciertos.
`Estadisticas`	% de aciertos, apostado, cobrado y rentabilidad de un jugador.
`CuentasCasaService` , `CuentasCasa`	Las cuentas de la casa (HU-54).
`AvisosApuestas`	Crea una notificación cuando una apuesta cambia de estado al resolverse.
`BajaService` , `BajaController`	Eliminar la cuenta (HU-18).
`MercadosController`	Página `/mercados` para apostar a largo plazo.

Paquetes `mercados` , `ligas` y `notificaciones`

Clase	Qué hace
`Mercado` , `Candidato`	Un mercado (nombre, deporte, fecha de cierre, estado, ganador) con sus candidatos y cuotas. Estados: `ABIERTO` , `CERRADO` , `RESUELTO` , `ANULADO` .
`MercadoService`	Crear mercados y candidatos, cambiar cuotas y cerrar.
`Liga` , `LigaService` , `LigasController`	Crear una liga con un código de 6 caracteres, unirse, ver su ranking, salir y borrarla.
`Notificacion` , `NotificacionService` , `NotificacionesController`	Avisos con texto, tipo, fecha y si se han leído; la campana de la cabecera muestra los pendientes.

Paquetes `gestion` , `api` , `config` y `web`

Clase	Qué hace
`GestionController` , `GestionService`	Panel `/gestion` : alta de competiciones, equipos y eventos; editar equipos (calidad, forma, escudo, jugadores); detalle de un evento con su resultado.
`GestionMercadosController` , `CasaController`	Gestión de mercados a largo plazo y página de cuentas de la casa.
`FootballDataCliente` , `FuenteDatosDeportivos`	Llama a la API. La interfaz permite cambiarla por otra fuente o por una falsa en las pruebas.
`PartidoApi` , `PlantillaApi`	La forma de los datos que devuelve la API.

`SincronizacionService`, `SincronizacionPlantillas`, `SincronizacionProgramada`	Convierten los datos de la API en eventos y jugadores nuestros; la última los lanza cada 30 minutos (partidos) y cada día (plantillas).
`SeguridadConfig`	Qué páginas son públicas y cuáles exigen sesión o rol (sección 23).
`RelojConfig`	El reloj de la aplicación, en hora de Madrid.
`DatosIniciales`	Al arrancar, crea los usuarios de prueba, competiciones, equipos, partidos reales de ejemplo y mercados (solo si la base de datos está vacía).
`UsuarioActualAdvice`	Pone en todas las páginas el usuario conectado, su saldo, el resumen del boleto y los avisos sin leer.
`Formato`, `Colores`	Formato de fechas e importes en las plantillas; colores para dibujar escudos cuando no hay imagen.

8. El modelo de datos

Estas son las entidades y cómo se relacionan. Cada una es una tabla (las relaciones "muchos a muchos" usan una tabla intermedia).

Entidad	Campos principales	Relaciones
Usuario	email, nombre, passwordHash, rol, saldo, ultimaRecarga, limiteDiario, limiteSemanal, pausaHasta, eliminado, version	Tiene muchas apuestas, notificaciones y ligas
Competicion	nombre, deporte	Muchos equipos (tabla `equipo_competiciones`)
Equipo	nombre, deporte, calidad, forma, escudoUrl, idExterno	Está en varias competiciones; tiene muchos jugadores
Jugador	nombre, posicion, dorsal, nacionalidad, fechaNacimiento, nota, idExterno	Pertenece a un equipo
Evento	fechaHora, estado, resultado, golesLocal, golesVisitante, fase, idExterno, suspendidoPorApi	Una competición, un equipo local y uno visitante
Apuesta	importe, cuota, estado, pagado, fecha	Un usuario; una o varias selecciones
Seleccion	pronostico (1X2), especial, cuota, estado	Una apuesta y **o bien** un evento **o bien** un candidato
Mercado	nombre, deporte, cierre, estado	Varios candidatos; un candidato ganador
Candidato	nombre, cuota	Un mercado; opcionalmente un equipo
Liga	nombre, codigo, creada	Un creador y muchos miembros (tabla `liga_miembro`)
Notificacion	texto, tipo, fecha, leida	Un usuario

Limites	importeMinimo, importeMaximo, maxSelecciones, juegoResponsable	Una sola fila para toda la aplicación

Ciclo de vida de un evento y de una apuesta

Evento

- **PROGRAMADO** → admite apuestas hasta su hora de inicio.
- **SUSPENDIDO** → no admite apuestas; se puede reactivar.
- **FINALIZADO** → tiene resultado; se puede corregir.
- **ANULADO** → no se juega; sus selecciones cuentan con cuota 1,00.

Apuesta

- **ACTIVA** → esperando resultados.
- **GANADA** / **PERDIDA** → resuelta y pagada.
- **ANULADA** → todas sus selecciones anuladas; se devuelve el importe.
- **CANCELADA** → la canceló el jugador antes de empezar.

Los diagramas UML completos (casos de uso, clases, estados, secuencias y modelo de datos) están en `docs/diseno/uml.md`. Están escritos en Mermaid, un formato de texto que GitHub dibuja solo.

9. Las páginas y sus direcciones

Dirección	Qué es	Quién
`/`	Portada con el partido destacado y accesos rápidos	Todos
`/eventos`	Catálogo por deporte, con buscador y cuotas	Todos
`/eventos/{id}/apostar`	Página del partido: cuotas, otros mercados, cara a cara y formulario de apuesta	Todos (apostar: jugador)
`/equipos`, `/equipos/{id}`	Listado de equipos y ficha de cada uno	Todos
`/mercados`	Apuestas a largo plazo	Todos (apostar: jugador)
`/ranking`, `/jugadores/{id}`	Ranking y perfil público	Todos
`/registro`, `/login`	Crear cuenta e iniciar sesión	Todos
`/boleto`	El boleto de la combinada	Jugador
`/apuestas`	Mis apuestas: activas, historial y estadísticas	Jugador
`/notificaciones`	Mis avisos	Con sesión
`/ligas`, `/ligas/{id}`	Mis ligas y el ranking de cada una	Jugador

`/cuenta`	Saldo, recarga, nombre, contraseña, juego responsable y baja	Con sesión
`/gestion/**`	Panel del creador: eventos, equipos, competiciones, mercados, límites, cuentas de la casa y sincronización	Creador

Las plantillas comparten trozos (**fragmentos**) que están en `templates/fragments/comun.html`: la cabecera con el menú y el saldo, el pie de página, la tarjeta de un partido con sus cuotas y el escudo. Así, si cambia la cabecera, se cambia en un solo sitio.

10. La configuración

Está en `src/main/resources/application.properties`. Lo más importante:

Propiedad	Para qué
`spring.datasource.url`	Qué base de datos usar. Por defecto, H2 en el archivo `./datos/apuestas`. En el servidor se cambia con la variable de entorno `SPRING_DATASOURCE_URL`.
`spring.flyway.locations`	`db/migration/{vendor}`: Flyway elige la carpeta `h2/` o `postgresql/` según la base de datos.
`spring.jpa.hibernate.ddl-auto=validate`	Hibernate no crea tablas: solo comprueba que las entidades coinciden con lo que ha creado Flyway. Si falta una columna, la aplicación no arranca y lo dice.
`apuestas.creador.password`, `apuestas.usuario.password`	Contraseñas de los usuarios de prueba (`creador123`, `usuario123`), cambiables con `CREADOR_PASSWORD` y `USUARIO_PASSWORD`.
`apuestas.saldo.*`	Bienvenida de 1.000, recarga de 200 cada 7 días.
`apuestas.api.*`	Dirección y clave de la API, competiciones (`PD` = LaLiga, `CL` = Champions), días hacia atrás (7) y hacia delante (21), intervalo (30 minutos).
`apuestas.zona-horaria`	`Europe/Madrid` (añadida en la revisión, fallo F-01).

Perfiles

Un **perfil** es un conjunto de ajustes que se activa al arrancar y se suma a los de `application.properties`:

- `application-compartir.properties`: para el túnel (sección 29). Desactiva la consola de H2.
- `application-nube.properties`: para Render. Desactiva la consola de H2 y exige `CREADOR_PASSWORD`.

La clave de la API nunca va en un archivo

`apuestas.api.token=${FOOTBALL_DATA_TOKEN:}` significa "léela de la variable de entorno `FOOTBALL_DATA_TOKEN`; si no existe, vacía". La clave es personal y el repositorio es visible para otros, así que se pone en IntelliJ (*Run → Edit Configurations → Environment variables*) o en Render, nunca en el código.

IV Cómo funciona cada cosa

Esta parte recorre cada funcionalidad: qué ve el usuario, qué reglas se aplican y en qué clases está.

11. Usuarios, sesión y moneditas

Registro (HU-11)

1. El formulario `/registro` pide email, nombre y contraseña. `RegistroForm` comprueba el formato del email, que no esté vacío y las longitudes.
2. `UsuarioService.crear` pasa el email a minúsculas (así `Ana@x.es` y `ana@x.es` son la misma cuenta) y comprueba que no exista ya.
3. La contraseña **nunca se guarda tal cual**: se cifra con **BCrypt**, que la convierte en un texto del que no se puede recuperar la original. Al iniciar sesión se cifra lo que se escribe y se compara.
4. El usuario empieza con **1.000 moneditas** de bienvenida (HU-14).

Inicio de sesión (HU-12)

Lo hace Spring Security con el formulario `/login`: `UsuarioDetailsService` busca el usuario por email y Spring compara la contraseña. Al entrar, `RecargaAlEntrarHandler` aplica la recarga si toca y lleva al usuario a la página que quería ver.

Saldo y recargas (HU-13, HU-14)

El saldo se ve siempre en la cabecera. Cada **7 días** desde la última recarga se suman **200 moneditas** gratis: se comprueba al iniciar sesión y al abrir *Mi cuenta* (`Usuario.aplicarRecargaPeriodica`). Las recargas no se acumulan: si pasan tres semanas sin entrar, se recibe una sola.

El saldo cambia con tres métodos de `Usuario`:

- `cargar(importe)`: al apostar. Si no hay saldo suficiente lanza `SaldoInsuficienteException`.
- `abonar(importe)`: al cancelar una apuesta o cobrar.
- `ajustar(diferencia)`: al resolver o corregir, suma o resta la diferencia con lo que ya se había pagado.

Mi cuenta (HU-47)

En `/cuenta` se cambia el nombre y la contraseña (pidiendo la actual), y están el juego responsable y la baja (sección 21).

Recuperar la contraseña (HU-17). En el inicio de sesión hay un enlace "¿Has olvidado tu contraseña?" que lleva a `/recuperar`:

- El jugador escribe su email. Si existe, `RecuperacionService` crea un código aleatorio y guarda solo su resumen SHA-256 en la tabla `token_recuperacion` (migración V10), con una caducidad de 30 minutos.

- `CorreoRecuperacion` envía el enlace por correo si el servidor tiene configurado el correo (variables `MAIL_*`); si no, lo escribe en el registro del servidor. La página responde lo mismo exista o no el email, para no revelar qué cuentas hay.
- El enlace lleva a `/recuperar/nueva`, donde se elige la contraseña nueva. El código solo sirve una vez; al usarlo se cierran las demás sesiones abiertas y se borran los intentos fallidos de inicio de sesión.

12. El algoritmo de cuotas

Una **cuota** es lo que se multiplica por el importe si se acierta: apostar 10 a cuota 1,80 devuelve 18 (8 de beneficio). Cuanto más probable es un resultado, más baja es su cuota. El algoritmo está en `cuotas/CalculadoraCuotas.java` y tiene cinco pasos:

Paso	Qué hace	Fórmula
1. Nivel	Cada equipo tiene una calidad (0–10, la pone el creador) y una forma (−2 a +2).	`nivel = calidad + 0,4 × forma`
2. Diferencia	Jugar en casa da ventaja.	`dif = nivelLocal + 0,5 − nivelVisitante`
3. Empate	Solo en fútbol. Es más probable cuanto más igualados estén.	`pE = max(0,10; 0,30 − 0,02 × |dif|)`

4. Reparto	El resto se reparte con una **curva logística**, que da casi todo al favorito cuando la diferencia es grande y la mitad a cada uno si es 0.	$r = 1 / (1 + e^{-0,45 \times dif})$
		`pL = (1 − pE) × r`
		`pV = (1 − pE) × (1 − r)`

5. Cuota	Se aplica el **margen de la casa** (1,07) y se redondea hacia abajo a 2 decimales, entre 1,01 y 50.	`cuota = 1 / (p × 1,07)`

Ejemplo resuelto

Local con calidad 8 y forma normal; visitante con calidad 7 y "mala racha" (−1).

- Niveles: local 8; visitante 7 − 0,4 = 6,6. Diferencia: 8 + 0,5 − 6,6 = **1,9**.
- Empate: 0,30 − 0,02 × 1,9 = **0,262**. Reparto: $r = 1 / (1 + e^{-0,855}) = 0,702$.
- Local: 0,738 × 0,702 = **0,518**. Visitante: 0,738 × 0,298 = **0,220**.
- Cuotas: local **1,80** · empate **3,56** · visitante **4,24**.

Si se suman las inversas, 1/1,80 + 1/3,56 + 1/4,24 = **1,072**. Que pase de 1 es lo que asegura que **la casa gana a la larga**: es el margen. El 6,5 % que aparece en las cuentas de la casa sale de 1 − 1/1,07.

Ajuste por volumen (HU-03, versión 2)

Si mucha gente apuesta a un resultado, su cuota baja y la de los demás sube, como en una casa real. La probabilidad se mezcla con el reparto del dinero:

```
peso = 0,30 × total / (total + 1.000)        (nunca llega a 0,30)
p'   = (1 − peso) × p + peso × (dinero en ese resultado / total)
```

En el ejemplo, con 1.000 moneditas apostadas al local, el peso es 0,15 y las cuotas pasan a **1,58 / 4,19 / 4,99**. Tras la revisión (fallos F-03 y F-13) cada jugador cuenta como máximo 100 moneditas por resultado y las combinadas se reparten entre sus selecciones, para que nadie pueda mover las cuotas a su favor.

Las cuotas no se guardan en el evento: se calculan cada vez que se muestran. La cuota que cuenta para una apuesta es la que se guarda en su selección al apostar.

13. Hacer una apuesta simple, paso a paso

Así recorre el código una apuesta de 10 moneditas a la victoria local:

1. **Vista.** En `apostar.html` el jugador elige "1" y escribe 10. Un poco de JavaScript enseña la ganancia potencial (18,00). El formulario envía `POST /eventos/{id}/apostar` con el pronóstico, el importe y la cuota que estaba viendo.
2. **Controlador.** `ApuestaController` valida el formulario (`ApuestaForm`) y llama a `ApuestaService.apostar` con el email del usuario conectado.
3. **Servicio** (todo dentro de una transacción):
   - Comprueba los **límites de la casa**: importe entre 1 y 500 y número de selecciones.
   - Comprueba que el usuario no es el creador.
   - Busca el evento y comprueba que **admite apuestas**: programado y sin haber empezado. Si no, `EventoNoDisponibleException`.
   - Calcula la cuota actual. Si es distinta de la que vio el jugador, `CuotasCambiadasException`: se le enseña la nueva para que decida.
   - Comprueba la cuota total máxima (1.000) y el **juego responsable** (pausa y límites personales).
   - `usuario.cargar(10)` descuenta el saldo y se guarda la apuesta con su selección.
4. **Respuesta.** Redirige a `/apuestas` con el mensaje "Apuesta realizada". Si algo ha fallado, vuelve al formulario con el error.

Cancelar y cambiar el importe (HU-26, HU-27)

Mientras **ninguno** de sus partidos haya empezado, una apuesta activa se puede cancelar (se devuelve todo el importe) o cambiar de importe (se cobra o devuelve la diferencia y se aplican las cuotas del momento). Lo deciden `Apuesta.cancelable`, `cancelar` y `cambiarImporte`.

14. El boleto y las combinadas (HU-28, HU-29, HU-30)

Al pulsar una cuota en el catálogo, la selección va al **boleto**, un objeto `Boleto` que vive en la sesión del jugador (`@SessionScope`): no se guarda en la base de datos hasta confirmar.

- No se puede repetir un partido: si se pulsa otro resultado del mismo partido, se sustituye.
- Como máximo 10 selecciones (límite de la casa, editable).

- **Cuota total = producto de las cuotas.** Ejemplo: 1,80 × 2,10 × 1,50 = **5,67**; con 10 moneditas se ganan 56,70.
- Con una sola selección, el boleto hace una apuesta simple.
- Al confirmar, si alguna cuota ha cambiado desde que se añadió, se enseñan las nuevas y hay que aceptarlas.

Una combinada se gana solo si se aciertan **todas** las selecciones; si un partido se anula, esa selección cuenta con cuota 1,00 y la combinada sigue con el resto.

15. Resolver las apuestas (HU-04, HU-05, HU-25)

Cuando el creador introduce el resultado (o llega de la API), `ResolucionService.introducirResultado`:

1. Marca el evento como `FINALIZADO` con su resultado.
2. Busca todas las selecciones de ese evento y, para cada una, llama a `apuesta.resolver(evento, resultado)`.
3. La selección queda `ACERTADA` o `FALLADA`, y la apuesta se **reevalúa**.
4. Si la apuesta ha cambiado de estado, `AvisosApuestas` crea un aviso para el jugador.

La regla de `Apuesta.reevaluar()`

- Si alguna selección ha fallado → **PERDIDA** (debe pagarse 0).
- Si no, y queda alguna pendiente → sigue **ACTIVA** (0).
- Si todas están anuladas → **ANULADA** (se devuelve el importe).
- Si no → **GANADA** (importe × producto de las cuotas, con 1,00 en las anuladas).

Después hace `usuario.ajustar(loQueDebePagar − loYaPagado)`. Gracias a esto, **corregir un resultado** funciona solo: si una apuesta pasa de ganada a perdida, se le quita al jugador lo que cobró; si pasa de perdida a ganada, se le paga.

Si el jugador ya se había gastado lo cobrado, su saldo puede quedar negativo. Se decidió mantenerlo y documentarlo (decisión 0002, `docs/decisiones/0002`; fallo F-12): mientras sea negativo no puede apostar, y la recarga semanal cubre la deuda.

Suspender un evento solo impide apostar. **Anular** pone sus selecciones como anuladas (cuota 1,00) y reevalúa las apuestas: en una simple se devuelve el importe.

16. Otros mercados de fútbol (HU-52)

En los partidos de fútbol, además del 1X2:

Mercado	Cómo se calcula la probabilidad	Ejemplo (el de antes)
Doble oportunidad (1X, X2, 12)	Suma de las probabilidades de los dos resultados	1,19 · 1,93 · 1,26

Más/menos de 2,5 goles	Cada equipo marca según una **distribución de Poisson** con media $1{,}3 \times e^{\pm 0{,}1 \times \text{dif}}$ (1,57 y 1,08 en el ejemplo). "Menos de 2,5" es la probabilidad de que haya 0, 1 o 2 goles en total	+2,5: 1,89 · −2,5: 1,84
Ambos marcan (sí/no)	P(local marca) × P(visitante marca)	Sí: 1,79 · No: 1,95

Para resolverlos hace falta el **marcador**: por eso en fútbol el creador introduce los goles de cada equipo y el resultado sale de ahí. Si solo se marca el ganador, la doble oportunidad se resuelve igual, pero las de goles y ambos marcan se anulan (cuota 1,00). Estos mercados no tienen ajuste por volumen.

17. Apuestas a largo plazo (HU-44, HU-45)

Un **mercado** es una pregunta a largo plazo ("¿Quién ganará el Balón de Oro?") con varios **candidatos**, cada uno con su cuota, que fija el creador a mano. Los jugadores apuestan en `/mercados` hasta la fecha de cierre. El creador puede cerrarlo antes, marcar el ganador (las apuestas se resuelven igual que las de un partido, con el mismo `reevaluar`), corregirlo o anularlo (se devuelve todo).

18. El panel del creador

Función	Detalles
Alta de competiciones y equipos (HU-01)	Con deporte y calidad 0–10. No se pueden repetir nombres.
Editar un equipo (HU-02, HU-50)	Calidad, forma, escudo y jugadores: añadir, quitar y poner nota.
Alta de eventos	Competición, local, visitante (distintos y de esa competición), fecha y fase.
Editar o borrar un evento (HU-46)	Solo si no ha empezado. Con apuestas no se pueden cambiar los equipos ni borrarlo: hay que anularlo.
Detalle de un evento (HU-04, HU-05, HU-06)	Dinero apostado a cada resultado y a cada tipo de apuesta; introducir o corregir el resultado o el marcador; suspender, reactivar o anular.
Límites de apuesta (HU-07)	Importe mínimo y máximo, selecciones por combinada e interruptor del juego responsable.
Mercados a largo plazo (HU-45)	Crear, añadir candidatos, cambiar cuotas, cerrar, marcar ganador, anular.
Cuentas de la casa (HU-54)	Apostado, pagado, beneficio y dinero en juego; margen real frente al teórico (6,5 %); desglose por tipo de apuesta y por deporte. Solo cuentan las apuestas cerradas.
Sincronizar ahora / Descargar plantillas	Lanza a mano la sincronización con la API.

19. Estadísticas de equipos

- **Ficha del equipo** (`/equipos/{id}` , HU-31): últimos 10 resultados, racha de los 5 últimos, balance de victorias, empates y derrotas, clasificación en cada competición (calculada con los partidos que tenemos: 3 puntos por victoria, 1 por empate) y próximos 5 partidos.
- **Gráfico de evolución** (HU-32): puntos acumulados partido a partido, dibujado en el servidor como SVG.
- **Plantilla y alineación** (HU-49): jugadores por posición con su nota; la alineación probable en 4-3-3 elige en cada posición a los de mejor nota.
- **Cara a cara** (HU-33): en la página del partido, la racha de cada equipo y los enfrentamientos anteriores entre los dos.

20. Ranking, perfiles, ligas y avisos

- **Ranking** (HU-36): por saldo, por ganancias (cobrado menos apostado en apuestas resueltas) o por % de aciertos (desde la revisión, con un mínimo de 5 apuestas resueltas). Tu posición sale destacada. No aparecen el creador ni las cuentas eliminadas.
- **Perfil público** (HU-48): puesto, saldo, aciertos y balance de otro jugador, sin enseñar sus apuestas.
- **Ligas privadas** (HU-51): quien crea la liga recibe un código de 6 caracteres sin letras que se confundan al dictarlas (sin 0/O ni 1/I/L). Los demás se unen escribiéndolo, da igual en mayúsculas o minúsculas. Cada liga tiene su ranking solo con sus miembros. Máximo 10 ligas por jugador; solo los miembros la ven; el creador puede borrarla y los demás salir.
- **Avisos** (HU-37): al ganar, perder o anularse una apuesta, aparece un aviso y la campana de la cabecera muestra cuántos hay sin leer.

21. Juego responsable y baja de la cuenta

- **Límites personales** (HU-10): el jugador fija cuánto puede apostar como mucho en 24 horas y en 7 días. Al apostar se suma lo apostado en ese periodo.
- **Pausa:** 1, 7 o 30 días sin poder apostar.
- El creador puede desactivar todo el juego responsable desde *Gestión → Límites*.
- **Eliminar la cuenta** (HU-18): pide la contraseña. Se cancelan las apuestas que aún se pueden cancelar, se borran los avisos, se borran sus ligas y sale de las demás, y la cuenta se **anonimiza**: el email pasa a `eliminado-<id>@apuestas.invalid`, el nombre a "Usuario eliminado", el saldo a 0 y la contraseña a una que nadie conoce. No se borra la fila, porque las apuestas resueltas siguen contando en las cuentas de la casa. La cuenta del creador no se puede eliminar.

22. Datos reales con la API

football-data.org es un servicio gratuito con datos de fútbol. Con una clave (`FOOTBALL_DATA_TOKEN`), la aplicación se sincroniza a los 30 segundos de arrancar y luego cada 30 minutos:

1. `FootballDataCliente` pide los partidos de cada competición configurada desde 7 días atrás hasta 21 días adelante.
2. `SincronizacionService` busca cada partido por su identificador de la API (`idExterno`). Si no existe, crea el evento, y los equipos si hace falta (con calidad 6,0, que el creador ajusta luego).
3. Según el estado que da la API:

Estado en la API	Qué hacemos
`SCHEDULED`, `TIMED`	Actualizar la hora. Si lo suspendió la propia API, reactivarlo.
`FINISHED`	Introducir el resultado con el marcador de los 90 minutos: se resuelven y pagan las apuestas.
`POSTPONED`, `SUSPENDED`	Suspender el evento.
`CANCELLED`	Anular el evento.
`IN_PLAY`, `PAUSED`	Nada, hasta que termine.

4. Cada partido va en su propia transacción: si uno falla, los demás se guardan y el error aparece en Gestión.

Plantillas (HU-50): a los 2 minutos de arrancar y una vez al día, `SincronizacionPlantillas` descarga los jugadores de los equipos que ya tenemos, de 7 ligas europeas (en el plan gratuito, la Champions no trae jugadores). Llegan sin dorsal y con nota 6,0. Los equipos de otras ligas se rellenan a mano.

El plan gratuito permite 10 peticiones por minuto y cada sincronización hace una por competición.

23. Seguridad

Medida	Cómo
Quién ve qué	`SeguridadConfig`: catálogo, equipos, mercados, ranking, perfiles, registro y login son públicos; `/gestion/**` solo para el rol `CREADOR`; el resto pide sesión.
Contraseñas	Cifradas con BCrypt; máximo 72 bytes.
CSRF	Cada formulario POST lleva un código secreto (token) que Thymeleaf añade solo. Así otra web no puede enviar formularios en nombre del usuario.
XSS	Thymeleaf escapa el texto (`th:text`): un nombre con `<script>` se ve como texto y no se ejecuta.
Datos de otros	Cada servicio comprueba que la apuesta, la liga o el aviso pertenecen al usuario conectado.
Fuerza bruta	Tras 5 contraseñas incorrectas, 15 minutos de bloqueo (desde la revisión).

Consola de la base de datos	Solo para el creador en local; desactivada en los perfiles públicos.
Recuperar la contraseña	Enlace de un solo uso que caduca a los 30 minutos; en la base de datos solo se guarda su resumen SHA-256.
Secretos	Clave de la API y contraseñas del servidor solo en variables de entorno.

V Base de datos

24. Flyway y las migraciones V1–V10

Al principio, Hibernate creaba las tablas solo a partir de las entidades. Funcionaba mientras la base de datos se borraba en cada arranque, pero en cuanto los datos se guardaron en un archivo apareció un problema: al añadir un campo a una entidad, las bases de datos que ya existían no tenían esa columna, y páginas como *Ranking* daban error 500. Por eso, en el Sprint 5 se pasó a **Flyway**.

Flyway funciona así:

- Cada cambio de la base de datos es un script SQL con un número: `V1__esquema_inicial.sql`, `V2__...`.
- Al arrancar, mira en una tabla suya (`flyway_schema_history`) cuáles ya se aplicaron y ejecuta solo los nuevos, en orden.
- Así cualquier base de datos antigua se pone al día sola y **sin perder datos**.

Migración	Qué cambia	Sprint
`V1__esquema_inicial`	Tablas `usuario`, `competicion`, `equipo`, `equipo_competiciones`, `evento`, `apuesta` y `seleccion`	5
`V2__identificadores_api`	`id_externo` en equipos y eventos, para enlazarlos con la API	5
`V3__forma_limites_largo_plazo`	Forma de los equipos; tablas `limites` (con su única fila: 1 / 500 / 10), `mercado` y `candidato`; la selección puede apuntar a un candidato en vez de a un evento	6
`V4__notificaciones`	Tabla `notificacion`	7
`V5__juego_responsable_y_bajas`	Límites diario y semanal, pausa y "eliminado" en el usuario	9
`V6__interruptor_juego_responsable`	Interruptor en `limites`	9
`V7__jugadores`	Tabla `jugador`	10
`V8__ligas_y_mas_apuestas`	Tablas `liga` y `liga_miembro`; goles en el evento; tipo especial en la selección	11
`V9__concurrencia_y_suspensiones`	`version` en el usuario (bloqueo optimista) y `suspendido_por_api` en el evento (revisión de fallos)	Revisión
`V10__recuperar_contrasena`	Tabla `token_recuperacion` y su secuencia, para recuperar la contraseña (HU-17)	Después

Dos carpetas y una regla de oro

- Los scripts están en `db/migration/h2/` (tu ordenador) y `db/migration/postgresql/` (el servidor). En PostgreSQL, `V8__esquema_postgresql.sql` crea de golpe todo lo de V1–V8, y a partir de ahí van los mismos scripts en las dos carpetas.
- Un cambio en una entidad necesita un script nuevo (`V11__...`) **en las dos carpetas**.
- **Nunca se modifica un script que ya está en `main`**: otros ordenadores ya lo han aplicado y Flyway detectaría el cambio y no arrancaría.

Como `ddl-auto=validate`, si se olvida una migración la aplicación no arranca y dice qué columna falta. Las pruebas lo detectan antes de juntar el cambio con `main`.

VI Calidad

25. Las pruebas automáticas

Una prueba automática es un programa que usa la aplicación y comprueba que el resultado es el esperado. Se ejecutan todas con `./mvnw test` (en Windows, `mvnw.cmd test`) en un par de minutos. Hay dos tipos:

- **Pruebas unitarias**: prueban una clase sola, sin base de datos ni web. Ejemplo: `CalculadoraCuotasTest` comprueba que la buena forma baja la cuota y que la suma de 1/cuota es mayor que 1.
- **Pruebas web** (las que terminan en `WebTest`): arrancan la aplicación completa con una base de datos de prueba y simulan peticiones con **MockMvc**, como si fuera un navegador. Ejemplo: inician sesión como Ana, apuestan 10 y comprueban que su saldo baja 10 y que la apuesta aparece en *Mis apuestas*.

La mayoría llevan `@Transactional`: todo lo que hacen se deshace al terminar, así cada prueba empieza con los mismos datos. Para la hora se usa un reloj fijo o la hora de Madrid (clase de apoyo `Hora`), así las pruebas dan lo mismo en cualquier ordenador.

Paquete	Clases de prueba (número de pruebas)
`api`	FootballDataClienteTest (2), SincronizacionServiceTest (13), SincronizacionPlantillasTest (5)
`apuesta`	ApuestaWebTest (10), CancelacionWebTest (3), CombinadaWebTest (11), ConcurrenciaSaldoTest (1), EliminarCuentaWebTest (5), EstadisticasTest (2), JuegoResponsableWebTest (10), LargoPlazoWebTest (12), LimitesWebTest (6), ModificarImporteWebTest (6), NotificacionesWebTest (5), OtrasApuestasWebTest (9), PerfilJugadorWebTest (4), ResolucionWebTest (10), RevisionFallosWebTest (23)
`config`	ContrasenasPruebaTest (1), DatosInicialesTest (7), PasswordCreadorFijaTest (1)
`cuotas`	CalculadoraCuotasTest (12), EspecialTest (5), FormaYVolumenWebTest (4)
`equipos`	AlineacionTest (5), EquipoTest (3), EquiposWebTest (4), PlantillaWebTest (8)
`eventos`	BusquedaWebTest (5), CatalogoWebTest (5), EventoTest (2), FichaEquipoWebTest (10)
`gestion`	CasaWebTest (2), EditarEventoWebTest (8), GestionWebTest (15)
`ligas`	LigasWebTest (7), LigasSinTransaccionTest (1)
`usuarios`	CuentaWebTest (6), RankingWebTest (2), RecuperacionWebTest (7), SesionesWebTest (1), UsuarioTest (4), UsuariosWebTest (7)
`web`	FormatoTest (5)

Total: 276 pruebas en 45 clases, contando `ApuestasApplicationTests` (2), que está en la raíz. ConcurrenciaSaldoTest, RevisionFallosWebTest, PasswordCreadorFijaTest y SesionesWebTest vienen de la revisión de fallos; RecuperacionWebTest, de HU-17.

26. La matriz de trazabilidad

`docs/requisitos/trazabilidad.md` enlaza cada **criterio de aceptación** de cada historia con la prueba que lo comprueba. Sirve para demostrar que lo que se pidió está hecho y comprobado.

45	94	91
historias hechas	criterios de aceptación	con prueba automática (97 %)

Los 3 criterios restantes se comprueban a mano porque dependen del navegador: la web en el móvil, la ganancia potencial calculada con JavaScript al escribir y la confirmación al eliminar la cuenta. Ningún criterio se queda sin prueba.

27. La integración continua

`.github/workflows/ci.yml` se ejecuta en GitHub en cada pull request y en cada cambio de `main`. Tiene dos trabajos:

- **build**: compila y pasa todas las pruebas con H2 (`./mvnw -B verify`).
- **postgresql**: arranca un PostgreSQL 17 y pasa las mismas pruebas contra él. Comprueba que las migraciones de PostgreSQL siguen al día con las entidades.

Si alguno falla, el pull request sale en rojo y no se debe juntar con `main`.

VII Despliegue

28. Ejecutarlo en tu ordenador

1. Instalar **Java 21** (por ejemplo, Temurin) y **Git**. Recomendado: **IntelliJ IDEA Community**.
2. Clonar: `git clone https://github.com/riverri/IS1.git`
3. Arrancar: en IntelliJ, abrir la carpeta y ejecutar `ApuestasApplication`; o en la terminal, `./mvnw spring-boot:run` (Windows: `mvnw.cmd spring-boot:run`).
4. Abrir `http://localhost:8080`.
5. Usuarios de prueba: `creador@apuestas.es` / `creador123` (creador) y `usuario@apuestas.es` / `usuario123` (jugador). También se puede crear una cuenta nueva.
6. Pruebas: `./mvnw test`.

La base de datos queda en la carpeta `datos/`. Para empezar de cero, se para la aplicación y se borra esa carpeta. Con la variable `FOOTBALL_DATA_TOKEN` se activan los partidos reales (sección 22).

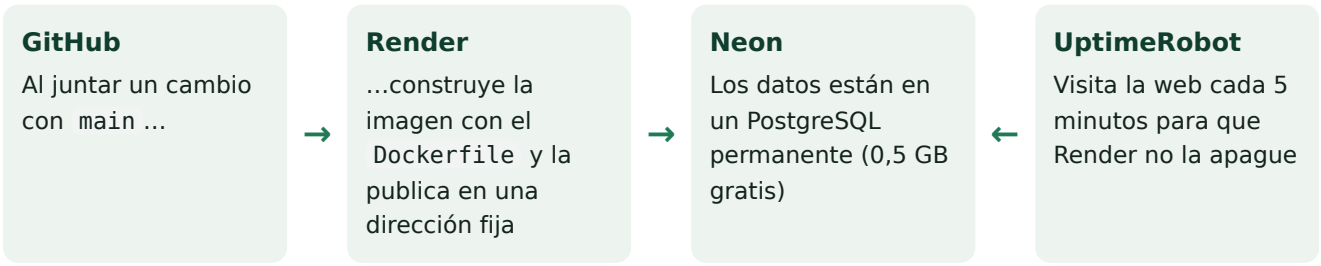
29. Compartirlo con un túnel

Para que otros jueguen desde su navegador mientras la aplicación corre en tu ordenador: `compartir.bat` (Windows) o `./compartir.sh`. Descarga **cloudflared**, arranca la aplicación con el perfil `compartir` y abre un túnel que da un enlace público del tipo `https://palabras-al-azar.trycloudflare.com`.

- El enlace solo funciona mientras el ordenador esté encendido con las dos ventanas abiertas, y cambia cada vez.
- El perfil desactiva la consola de H2. Desde la revisión, también pide una contraseña nueva para el creador, porque la del README es pública.
- Si la red de la facultad bloquea el túnel, se puede usar los datos del móvil.

30. El servidor permanente

Para tenerla siempre en Internet sin un ordenador encendido se usan tres servicios gratuitos:



- El `Dockerfile` tiene dos fases: compila con Maven y luego copia solo el programa a una imagen con Java, con el perfil `nube`, la hora de Madrid y la memoria ajustada a los 512 MB del plan gratuito.

- En Render se configuran las variables: `CREADOR_PASSWORD` (obligatoria), `SPRING_DATASOURCE_URL`, `_USERNAME` y `_PASSWORD` (los de Neon) y, si se quiere, `FOOTBALL_DATA_TOKEN`.
- Render apaga las aplicaciones gratuitas tras 15 minutos sin visitas (y la siguiente tarda un minuto en arrancar); por eso se usa UptimeRobot.
- No se usa la base de datos gratuita de Render porque se borra a los 30 días.

VIII El proceso Scrum

Scrum es una forma de organizar el trabajo en equipo en ciclos cortos de duración fija (**sprints**). Al final de cada sprint hay algo que funciona y se puede enseñar. Todo el proceso del proyecto está descrito en `docs/scrum/proceso.md`, basado en el Tema 3 de la asignatura.

31. Roles, eventos y artefactos

Roles

Rol	Quién	Qué hace
Product Owner	El profesor (también hace de cliente)	Decide qué se construye y en qué orden: es el responsable del Product Backlog.
Scrum Master	Carlos Jurado (issue #28)	Vela por que se siga Scrum, organiza los eventos y quita obstáculos.
Equipo de desarrollo	Los cinco miembros	Decide cuánto trabajo cabe en el sprint y cómo hacerlo.

El profesor figura como Product Owner desde el PR #33. Carlos Jurado es el Scrum Master desde el 16 de octubre.

Eventos

Evento	Cuándo y para qué
Sprint Planning	Al empezar el sprint: se elige el objetivo (**Sprint Goal**) y las historias que entran (**Sprint Backlog**).
Daily Scrum	Cada día, 15 minutos: qué hice, qué haré, qué me bloquea.
Sprint Review	Al final: se enseña lo hecho al Product Owner, que lo acepta o no.
Retrospectiva	Tras la review: qué fue bien, qué mejorar y al menos una mejora concreta para el siguiente sprint.

Cada sprint tiene su **acta** en `docs/scrum/sprints/sprint-NN.md` con la planning, los dailies, la review y la retrospectiva.

Artefactos en GitHub

- **Product Backlog**: `docs/scrum/product-backlog.md`, con las 35 filas del documento de backlog del equipo, ordenadas y priorizadas.
- **Sprint Backlog**: desde el Sprint 12, issues con la etiqueta del sprint en el tablero de GitHub Projects (columnas: Product Backlog → Sprint Backlog → En curso → En revisión → Terminado, con un máximo de 1 tarea en curso por persona).
- **Incremento**: la rama `main` al terminar el sprint.

Definition of Ready y Definition of Done

Ready: una historia puede entrar en un sprint si…

- Sigue el formato *Como... quiero... para...*
- Tiene criterios de aceptación *Dado... cuando... entonces...*
- Está estimada y mide 13 puntos o menos.
- Sus dependencias están hechas.

Done: una historia está terminada si…

- Cumple sus criterios de aceptación.
- Está en `main` por un PR revisado.
- Tiene pruebas automáticas que pasan.
- Está en la matriz de trazabilidad.
- Se ve bien en móvil y ordenador.
- La documentación está al día.
- El PO la ha aceptado en la review.

Estimación y priorización

- **Puntos de historia** en escala Fibonacci (1, 2, 3, 5, 8, 13, 20): miden el tamaño relativo, no horas. Se estiman con **Planning Poker**.
- **Velocidad**: puntos terminados por sprint; sirve para planificar el siguiente.
- **MoSCoW**: cada fila del backlog es Must (imprescindible), Should, Could o Won't (esta vez no). El **MVP** (producto mínimo viable) son los Must.

32. Product Backlog e historias de usuario

Las historias están en `docs/requisitos/historias-de-usuario.md`, agrupadas por tema, cada una con la fila del backlog de la que sale y sus criterios de aceptación. Hay 53 escritas:

Estado	Historias
Hechas (45)	HU-01 a HU-14, HU-17 a HU-37, HU-44 a HU-52 y HU-54
Descartadas, Won't (8)	HU-15 y HU-16 (cuenta bancaria y retiradas: chocan con "sin dinero real") y HU-38 a HU-43 (seguir usuarios, feed, privacidad, copiar apuestas). En su lugar se hizo el perfil público (HU-48) y las ligas (HU-51).

Del backlog de 35 filas: 33 hechas (la 22 se completó con HU-17 después del Sprint 12) y las 25 y 26 descartadas. Cada historia tiene además una prioridad del 1 (muy alta) al 5 (muy baja), revisada por el equipo el 16 de octubre. Las filas 1–18 son Must y forman el MVP, que quedó completo en el Sprint 4.

33. Los doce sprints

Sprint	Objetivo	Historias y tareas (puntos)	Pts.
1	Catálogo visible; registro, sesión y moneditas; el creador da de alta competiciones, equipos y eventos	HU-01 (8), HU-11 (3), HU-12 (3), HU-13 (2), HU-14 (2), HU-08 (2), HU-19 (3)	23

2	Cuotas propias y apuestas simples	HU-03 (8), HU-20 (3), HU-23 (5), HU-24 (3), editar equipos (2); añadido durante el sprint: HU-22 (3), HU-26 (2), HU-36 (3), más deportes (3), escudos (1)	33
3	Resolución automática, historial y estadísticas	HU-04 (5), HU-25 (5), HU-05 (3), HU-06 (2), HU-34 (2), HU-35 (3), HU-36 (2), largo plazo al backlog (2)	24
4	Boleto y combinadas (MVP completo)	HU-28 (5), HU-29 (5), HU-30 (5), HU-07 parcial (1), adaptar el modelo (2)	18
5	Partidos y resultados reales desde la API	HU-21 (8), HU-25 con la API (3), escudos de la API (1), HU-05 desde la API (1); añadido: migraciones con Flyway (3)	16
6	Forma y volumen en las cuotas; límites; largo plazo	HU-02 (2), HU-03 v2 (3), HU-07 (2), HU-44 (5), HU-45 (5), V3 (1)	18
7	Ficha de equipo, cara a cara, avisos, cambiar importe	HU-31 (5), HU-33 (3), HU-37 (5), HU-27 (2), V4 (1)	16
8	Corregir eventos, mi cuenta, perfiles; UML y métricas	HU-46 (3), HU-47 (2), HU-48 (2), UML (3), métricas (2)	12
9	Evolución de equipos, juego responsable, baja	HU-32 (3), HU-10 (5), HU-18 (3), V5 y V6 (1); añadido: interruptor (1), rediseño visual HU-09 (3)	16
10	Plantillas y alineación probable	HU-49 (5), HU-50 (5), V7 (1)	11
11	Ligas privadas, más apuestas, cuentas de la casa	HU-51 (5), HU-52 (8), HU-54 (3), V8 (1)	17
12	Servidor permanente, trazabilidad y tablero	#24 trazabilidad (3), #25 dinero por tipo de apuesta (2), #26 PostgreSQL (5); #28 Scrum Master (resuelto: Carlos Jurado) y #29 retrospectivas pendientes	10

Ojo con las fechas

Las actas dicen que los sprints duran dos semanas, pero los sprints 1 a 9 figuran cerrados "antes de plazo" el 1 y el 2 de octubre, y los 10 a 12 tienen fechas de dos semanas (2/10–16/10, 16/10–30/10, 30/10–13/11). La wiki del repositorio, en cambio, habla de un Sprint 1 real del 19/09 al 3/10. Conviene que el equipo decida cómo cuadrar las actas con el calendario real (sección 38). Además, las reviews y retrospectivas están sin rellenar (issue #29), y en las actas el único responsable que figura es Jing Li.

34. Métricas

`docs/scrum/metricas.md` tiene la **velocidad** (puntos comprometidos y completados por sprint) y el **burnup** (puntos acumulados), con gráficas que genera `docs/scrum/graficas/generar.py`.

- Sprints 1–11: 185 puntos comprometidos y 204 completados (19 los añadió el PO durante los sprints). Con el Sprint 12, 214.
- Media: unos 18 puntos por sprint. En cada sprint se completó todo lo comprometido.
- Acumulado: 23 → 56 → 80 → **98 (MVP, Sprint 4)** → 114 → 132 → 148 → 160 → 176 → 187 → 204 → 214.
- No hay burndown diario porque los sprints se hicieron concentrados en una o dos sesiones; el documento explica cómo llevarlo a partir de ahora.

IX Historia y revisión

35. Cronología del desarrollo

Fecha	Qué se hizo
1 oct 2026	Se crea el repositorio con la estructura de Scrum, se incorpora el Product Backlog del equipo y se alinea el proceso con la teoría. Esqueleto de Spring Boot. Sprints 1 a 7: catálogo, cuentas, gestión, partidos reales, cuotas, apuestas simples, resolución, combinadas, API, Flyway, más deportes, túnel para compartir, forma y volumen, largo plazo, ficha de equipo, avisos. Pull requests #1 a #14.
2 oct	Sprints 8 a 12: editar eventos, cuenta y perfiles, UML y métricas, evolución, juego responsable, baja, rediseño visual, plantillas, ligas, más tipos de apuesta, cuentas de la casa. Publicación en Render (#23), PostgreSQL y trazabilidad (#30). Issues #24 a #29 y tablero.
3–4 oct	Roles corregidos: el profesor es el Product Owner (#33). Respuesta a los issues #31 (invitar al profesor) y #32 (documentación), abiertos por un compañero. Se crea IS1-Copy con todo el historial.
4 oct	Informe del proyecto y revisión completa del código: 34 fallos. Se corrigen todos en IS1-Copy (PR #1 de IS1-Copy), con 34 pruebas nuevas.
5 oct	Los 34 fallos se dan de alta como issues en IS1 (#34 a #67).
6 oct	Historias de usuario con criterios de aceptación en formato Dado / Cuando / Entonces. Los 34 arreglos de IS1-Copy se pasan a IS1 (PR #70) y se cierran los issues #34–#67. Wiki del proyecto, publicada sola desde `docs/wiki` (PR #71). Recuperar la contraseña, HU-17 (PR #72).
16 oct	Plan de gestión de riesgos en la wiki; Carlos Jurado, Scrum Master (PR #76). El equipo prioriza las historias del 1 al 5 y propone ideas para la ampliación (discusión #74).
19 oct	El proyecto pasa a llamarse **Better** (PR #79) y se sube la presentación del estado del proyecto (PR #80).

Cada cambio se hizo en una rama y entró en `main` por un pull request con el CI en verde, fusionado con *squash* (todos los commits del PR quedan como uno solo).

36. IS1 e IS1-Copy

riverri/IS1 es el repositorio del proyecto: ahí está todo y es el que hay que usar. **riverri/IS1-Copy** es una copia completa que se hizo el 4 de octubre para revisar el código y corregir los fallos sin tocar el principal.

	riverri/IS1	riverri/IS1-Copy
Uso	El repositorio principal; se sigue desarrollando aquí	Copia de trabajo de la revisión; ya no se usa

Fallos	Los 34 corregidos (PR #70); issues #34–#67 cerrados	Donde se corrigieron primero (PR #1 de IS1-Copy)
Después	Recuperar la contraseña, wiki, riesgos, nombre Better…	Nada: se quedó como estaba tras la revisión

37. La revisión: 34 fallos

Se partió de un informe del proyecto con 14 fallos (**F-01 a F-14**) y una revisión completa del código encontró 20 más (**N-01 a N-20**). Todos están corregidos en IS1 (PR #70), y cada uno tiene su issue cerrado con la explicación, cómo reproducirlo y cómo se arregló.

Gravedad: **alta** puede tumbar la aplicación, perder dinero o dejar entrar a quien no debe · **media** comportamiento incorrecto · **baja** molestia o caso raro.

Del informe

F-01 · El reloj usa la zona del servidor alta issue #34

El servidor de Render está en hora UTC y los partidos en hora de Madrid, así que se podía apostar hasta 2 horas después de empezar un partido. **Arreglo:** reloj fijo en Europe/Madrid y `TZ` en el Dockerfile.

F-02 · El saldo no tiene control de concurrencia alta #35

Dos apuestas a la vez (doble clic) podían leer el mismo saldo y una pisar a la otra. **Arreglo:** bloqueo optimista con `@Version`: si dos operaciones chocan, la segunda se rechaza con un aviso.

F-03 · Cuotas manipulables apostando y cancelando media #36

Como cancelar es gratis, alguien con mucho saldo podía mover las cuotas y luego cancelar. **Arreglo:** cada jugador cuenta como mucho 100 por resultado en el volumen.

F-04 · El perfil compartir mantenía la contraseña pública del creador alta #37

`creador123` está en el README y el túnel hace pública la aplicación. **Arreglo:** en los perfiles públicos la contraseña es obligatoria y los scripts la piden.

F-05 · Un partido con error bloqueaba toda la sincronización alta #38

Todo iba en una transacción: un error deshacía todos los resultados. **Arreglo:** una transacción por partido; los errores se apuntan y los demás se guardan.

F-06 · Sin tope de cuota total en combinadas media #39

Diez selecciones de cuota alta daban una ganancia que no cabía en la base de datos (error 500). **Arreglo:** la cuota total no puede pasar de 1.000.

F-07 · Prórroga y penaltis mal resueltos media #40

En las apuestas cuenta el resultado de los 90 minutos; se usaba el ganador final. **Arreglo:** se usa el marcador de los 90 minutos de la API, y si no lo da, el partido queda para resolverlo a mano.

F-08 · Sin límite de intentos de inicio de sesión media #41

Se podían probar contraseñas sin fin. **Arreglo:** 5 fallos seguidos bloquean el email 15 minutos.

F-09 · El ranking cargaba todas las apuestas en memoria media #42

Cada vez más lento. **Arreglo:** una consulta que suma en la base de datos y devuelve una fila por jugador.

F-10 · % de aciertos sin mínimo de apuestas baja #43

1 de 1 (100 %) quedaba por delante de 40 de 50. **Arreglo:** mínimo de 5 apuestas resueltas.

F-11 · Añadir al boleto con un enlace GET baja #44

Un GET no debe cambiar datos. **Arreglo:** formulario POST con token CSRF.

F-12 · Saldo negativo al corregir un resultado baja #45

No es un fallo de código sino una decisión: se mantiene y se documenta en `docs/decisiones/0002` (sección 15).

F-13 · Volumen y cuentas eliminadas media #46

Las combinadas contaban el importe entero en cada selección y las cuentas eliminadas seguían recibiendo ajustes. **Arreglo:** el importe se reparte entre las selecciones y las cuentas eliminadas no se tocan.

F-14 · Consola de H2 abierta alta #47

Cualquiera podía entrar en la base de datos desde el navegador. **Arreglo:** solo el creador.

De la revisión del código

N-01 · Un nombre repetido impedía arrancar alta #48

Dos equipos con el mismo nombre hacían fallar los datos iniciales. **Arreglo:** no se permiten nombres repetidos y la búsqueda tolera los antiguos.

N-02 · Un evento anulado bloqueaba la sincronización alta #49

Arreglo: la sincronización no toca los eventos anulados.

N-03 · El creador podía apostar alta #50

Y luego introducir él mismo el resultado. **Arreglo:** se comprueba el rol y se ocultan los botones.

N-04 · Partidos duplicados al reiniciar media #51

Un partido de ejemplo movido o borrado reaparecía. **Arreglo:** los datos de ejemplo solo se cargan en una base de datos vacía.

N-05 · La apuesta simple no avisaba del cambio de cuota media #52

Arreglo: se envía la cuota vista y, si ha cambiado, se avisa.

N-06 · La API deshacía las suspensiones del creador media #53

Arreglo: se marca quién suspendió; la API solo reabre lo que suspendió ella.

N-07 · Inyección de código con el nombre de un jugador media #54

Un apóstrofo rompía la confirmación al quitar un jugador. **Arreglo:** el nombre va en un atributo `data-`, no dentro del JavaScript.

N-08 · Se podía bloquear la baja de otro media #55

Registrándose con el email anónimo que tendría su cuenta. **Arreglo:** el dominio `@apuestas.invalid` está reservado.

N-09 · Contraseñas largas con tildes daban error 500 media #56

BCrypt admite 72 bytes y las tildes ocupan 2. **Arreglo:** se comprueban los bytes y se avisa.

N-10 · Las otras sesiones seguían abiertas media #57

Arreglo: al cambiar la contraseña o darse de baja se cierran las de otros navegadores.

N-11 · Al corregir se ignoraba el ganador elegido media #58

Arreglo: un marcador que contradice el resultado se rechaza.

N-12 · "Aceptar nuevas cuotas" había que pulsarlo dos veces baja #59

N-13 · Página 404 si un partido del boleto empezaba baja #60

N-14 · La liga de una cuenta eliminada no se podía borrar baja #61

Arreglo: evento `CuentaEliminada` que borra sus ligas.

N-15 · Nombres o emails demasiado largos daban error 500 baja #62

N-16 · Una nota "NaN" se guardaba como 0 baja #63

N-17 · Un equipo inexistente daba 500 en vez de 404 baja #64

N-18 · Plantillas asignadas a un equipo equivocado baja #65

N-19 · Sin tiempo máximo de espera con la API baja #66

Arreglo: 5 s para conectar y 20 s para responder.

N-20 · Se marcaban como leídos avisos que no se veían baja #67

Dos cosas del informe se dejaron como están a propósito: avisar en el registro de que un email ya existe (lo pide HU-11) y recuperar la contraseña, que era una funcionalidad nueva (HU-17, #27) y no un fallo; se hizo después (PR #72).

X Para el día a día

38. Qué queda por hacer

Tarea	Dónde
Elegir las ideas de la ampliación	Discusión #74 de GitHub. Decidirlas con el Product Owner y escribir sus historias con prioridad y criterios de aceptación.
Repartir las épicas	Un responsable por épica que escriba sus historias, haga sus diagramas UML y revise sus pull requests.
Rellenar las reviews y retrospectivas	Issue #29 y actas de los sprints (secciones "Sprint Review" y "Retrospectiva").
Cuadrar las actas con el calendario real	La wiki dice que el Sprint 1 va del 19/09 al 3/10; las actas tienen otras fechas (sección 33). Hay que decidir si se reorganizan.
Repartir el trabajo y que se vea	A partir de ahora, cada persona con su rama, sus commits y sus PR revisados por otro (sección 39). Es lo que muestra la participación de cada uno.

39. Cómo hacer un cambio

El flujo está en `CONTRIBUTING.md`. Paso a paso, por ejemplo para el issue #63 (nota NaN):

1. En el tablero, asignarse el issue y moverlo a **En curso**.
2. Actualizar `main` y crear una rama:

   ```
   git checkout main
   git pull
   git checkout -b fix/63-nota-nan
   ```

3. Hacer el cambio (en este caso, en `equipos/Jugador.java`) y añadir una prueba (por ejemplo, en `PlantillaWebTest`).
4. Comprobar que todo pasa: `./mvnw test`.
5. Guardar el cambio con un mensaje en el formato del proyecto (`feat:`, `fix:`, `docs:`, `test:` …):

   ```
   git add .
   git commit -m "fix: rechaza la nota NaN de un jugador (#63)"
   git push -u origin fix/63-nota-nan
   ```

6. En GitHub, abrir un **pull request** hacia `main` con `Closes #63` en la descripción (así el issue se cierra solo al fusionar).
7. Esperar a que el CI salga en verde y a que un compañero lo revise y apruebe.
8. Fusionar con *Squash and merge* y borrar la rama.

Si el cambio toca una entidad

Añadir un script `V11__...sql` en `db/migration/h2/` **y** en `db/migration/postgresql/`. No modificar nunca un script que ya esté en `main`.

40. Preguntas que te pueden hacer

¿Qué arquitectura tiene la aplicación?
Una aplicación web Spring Boot por capas (MVC): plantillas Thymeleaf para la vista, controladores que reciben las peticiones, servicios con las reglas del negocio y transacciones, y entidades JPA con sus repositorios sobre H2 o PostgreSQL. El código está organizado por funcionalidad en 12 paquetes.

¿Cómo se calculan las cuotas?
Nivel de cada equipo = calidad + 0,4 × forma; diferencia con ventaja de 0,5 para el local; probabilidad de empate según lo igualados que estén; el resto con una curva logística; ajuste por el dinero apostado (hasta un 30 %); y cuota = 1 / (probabilidad × 1,07). El 1,07 es el margen de la casa (sección 12).

¿Cómo se asegura que la casa no pierde a la larga?
Con el margen: la suma de 1/cuota de un evento siempre es mayor que 1, porque las probabilidades se multiplican por 1,07 y la cuota se redondea hacia abajo. La página de cuentas de la casa compara el margen real con el teórico (6,5 %).

¿Qué pasa si se corrige un resultado ya pagado?
Las apuestas se reevalúan y el saldo se ajusta con la diferencia entre lo que debe cobrar ahora y lo que ya cobró. El saldo puede quedar negativo; es una decisión documentada en `docs/decisiones/0002` (fallo F-12, #45).

¿Por qué Flyway?
Porque las bases de datos guardadas no tenían las columnas nuevas al cambiar las entidades y la aplicación fallaba. Con Flyway cada cambio es un script numerado que se aplica una vez y en orden, y los datos se conservan.

¿Cómo sabéis que funciona?
Con 276 pruebas automáticas, que se ejecutan en cada pull request con H2 y con PostgreSQL, y con la matriz de trazabilidad, que enlaza los 94 criterios de aceptación de las historias hechas con su prueba (91 automáticos y 3 a mano).

¿Qué es una transacción y dónde la usáis?
Un grupo de cambios en la base de datos que se hace entero o no se hace. Por ejemplo, al apostar se descuenta el saldo y se guarda la apuesta: si algo falla, no se descuenta nada. Se marcan con `@Transactional` en los servicios. En la sincronización con la API se usa una por partido para que un fallo no deshaga los demás.

¿Cómo protegéis las contraseñas y las páginas?
Las contraseñas se guardan cifradas con BCrypt. Spring Security decide qué páginas son públicas, cuáles exigen sesión y cuáles el rol de creador; los formularios llevan token CSRF; y tras 5 contraseñas incorrectas la cuenta se bloquea 15 minutos.

¿Qué es el MVP y cuándo lo tuvisteis?
El producto mínimo viable: las filas Must del backlog (catálogo, cuentas, moneditas, cuotas, apuestas simples y combinadas, resolución y panel del creador). Quedó completo en el Sprint 4, con 98 puntos.

¿Qué historias no se han hecho y por qué?
Todas las que se eligieron están hechas, también HU-17 (recuperar la contraseña). HU-15 y HU-16 (cuenta bancaria) se descartaron porque la aplicación no usa dinero real, y HU-38 a HU-43 (parte social completa) por su coste; en su lugar se hicieron el perfil público y las ligas privadas.

¿De dónde salen los partidos reales?
De la API de football-data.org: cada 30 minutos se descargan los partidos de LaLiga y la Champions, y al terminar uno se resuelven sus apuestas. La clave va en una variable de entorno, nunca en el código.

¿Cómo está publicada?
En Render, que construye la imagen Docker desde GitHub al fusionar en `main`, con la base de datos en Neon (PostgreSQL) y UptimeRobot para que no se apague.

¿Qué fallos encontrasteis en la revisión?
34: los más graves eran la hora del servidor (se podía apostar con el partido empezado), el saldo sin control de concurrencia, la contraseña pública del creador en el túnel, la sincronización que se bloqueaba con un partido erróneo y que el creador pudiera apostar. Están en los issues #34–#67 (sección 37).

41. Glosario

Término	Significado
1X2	Apuesta al resultado: 1 gana el local, X empate, 2 gana el visitante.
API	Servicio al que otro programa pide datos por Internet (aquí, football-data.org).
BCrypt	Algoritmo para cifrar contraseñas de forma que no se puedan recuperar.
Boleto	Lista de selecciones que el jugador va juntando para una combinada.
CI (integración continua)	Compilar y pasar las pruebas automáticamente en cada cambio.
Combinada	Apuesta con varias selecciones de partidos distintos; se gana si se aciertan todas.
Commit	Un cambio guardado en Git, con su descripción y su autor.
CSRF	Ataque en el que otra web envía un formulario en tu nombre; se evita con un token.
Cuota	Número por el que se multiplica el importe si se acierta.
Entidad	Clase Java que corresponde a una tabla de la base de datos.
Flyway	Herramienta que aplica en orden los scripts de la base de datos.

Issue	Tarea, historia o fallo apuntado en GitHub.
JPA / Hibernate	Traducción automática entre objetos Java y tablas.
Margen de la casa	Lo que se queda la casa a la larga (1,07 → 6,5 %).
Mercado	Apuesta a largo plazo con varios candidatos.
Migración	Script SQL numerado que cambia la estructura de la base de datos.
MockMvc	Herramienta de pruebas que simula peticiones web sin navegador.
MoSCoW	Priorización: Must, Should, Could, Won't.
MVC	Modelo-Vista-Controlador: separar datos, presentación y control.
MVP	Producto mínimo viable: lo imprescindible para que sirva.
Perfil (Spring)	Conjunto de ajustes que se activa al arrancar (`compartir` , `nube`).
Pull request (PR)	Propuesta para juntar una rama con `main` , que se revisa antes.
Rama	Copia de trabajo del código donde se hace un cambio sin tocar `main` .
Repositorio (JPA)	Interfaz para buscar y guardar entidades.
Selección	Un pronóstico dentro de una apuesta, con su cuota.
Servicio	Clase con las reglas del negocio.
Sesión	Memoria que el servidor guarda para cada usuario conectado.
Sprint	Ciclo de trabajo de duración fija en Scrum.
Squash	Fusionar un PR juntando todos sus commits en uno.
Thymeleaf	Plantillas HTML que se rellenan con datos de Java.
Transacción	Grupo de cambios que se hace entero o no se hace.
Trazabilidad	Enlazar cada requisito con la prueba que lo comprueba.
Velocidad	Puntos de historia terminados por sprint.
XSS	Ataque que mete código en una página a través de un texto; se evita escapando el texto.